

Reviewer Pack — Minimal Rank of Quadratic Entanglement Patterns vs BP_ReadTw...

Entry 04eb145028b1 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-24 23:14:42 UTC

Minimal Rank of Quadratic Entanglement Patterns vs BP_ReadTwice Circuit Depth

Entry ID: 04eb145028b1 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-24 23:14:42 UTC

1. Conjecture

Field A (mathematical branch): Quantum Information Theory (Entanglement Theory) **Field B** (complexity object): Complexity Theory: BP_ReadTwice Complexity

Statement:

[‘For a given n -qubit system, the minimal rank of its quadratic entanglement pattern is polynomially related to the depth of its corresponding BP_ReadTwice circuit.’, ‘Specifically, there exists a constant c such that for all n -qubit systems, the minimal rank of the quadratic entanglement pattern is at most c times the depth of any BP_ReadTwice circuit that computes it.’, ‘Equivalently, for every n , there exists a function $f(n)$ such that for every n -qubit system, if an instance requires a BP_ReadTwice circuit of depth d to compute its quadratic entanglement pattern, then the minimal rank is at most $f(n)d$.’]

Rationale (proposer’s reasoning):

[‘Quadratic entanglement patterns provide a concise description of the entanglement structure in quantum systems. A deeper understanding of these patterns may shed light on the complexity of computing them using classical circuits.’, ‘The proposed connection suggests that the difficulty of computing quadratic entanglement patterns might be captured by the BP_ReadTwice circuit model, which is known to be a powerful resource for describing complex computations.’, ‘This conjecture bridges quantum information theory with computational complexity, potentially revealing new insights into both fields.’]

Taxonomy category: QuantumEntanglementToBPTwice (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): fa67bbd64fc6c44e

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if Spearman’s rank correlation coefficient (ρ) between minimal rank and BP_ReadTwice circuit depth is $\rho \geq 0.7$ for at least 30 seeds, or if there

exists a constant $c \leq 5$ such that the minimal rank is less than or equal to c times the BP_ReadTwice circuit depth for all seeds.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL PROOFS	SAFE	0.70	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - minimal rank AND quadratic entanglement AND BP_ReadTwice complexity - entanglement theory AND polynomial relationship AND BP_ReadTwice depth - quadratic entanglement pattern AND n-qubit system AND circuit depth

Top relevant hits considered: - [<http://arxiv.org/abs/2412.14709v1>] Minimal rank of primitively n -universal integral quadratic forms over local rings - [<http://arxiv.org/abs/2011.13195v2>] Average rank of quadratic twists with a rational point of almost minimal height - [<http://arxiv.org/abs/2507.23338v1>] Extending bounds on minimal ranks of universal quadratic lattices to larger number fields - [<http://arxiv.org/abs/2111.08018v2>] Entanglement dynamics in hybrid quantum circuits - [<http://arxiv.org/abs/2205.15383v5>] Interacting CFTs for all couplings: Thermal versus Entanglement Entropy at Large N - [<http://arxiv.org/abs/2502.18759v1>] Some permutation polynomials via linear translators - [<http://arxiv.org/abs/2011.11054v2>] Consecutive Quadratic Residues And Quadratic Nonresidue Modulo p - [<http://arxiv.org/abs/2505.13061v4>] 3D Visual Illusion Depth Estimation

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.4s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_random_qubit_system(n):
        # Generate a random n-qubit system using quantum circuits (simplified)
        return [random.choice([0, 1]) for _ in range(2**n)]

    def compute_quadratic_entanglement_pattern(qubits):
        # Compute the quadratic entanglement pattern (simplified)
        n = int(math.log2(len(qubits)))
        pattern = [[qubits[i] * qubits[j] for j in range(n)] for i in range(n)]
        return sum(sum(row) for row in pattern)
```

```

def construct_bp_readtwice_circuit(pattern):
    # Construct the corresponding BP_ReadTwice circuit (simplified)
    n = int(math.log2(len(pattern)))
    depth = 0
    for _ in range(n):
        depth += 1
    return depth

n = random.randint(5, 40)
qubits = generate_random_qubit_system(n)
pattern = compute_quadratic_entanglement_pattern(qubits)
depth = construct_bp_readtwice_circuit(pattern)

if depth == 0:
    return {
        "metric_name": "Minimal Rank",
        "metric_value": None,
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

minimal_rank = pattern

return {
    "metric_name": "Minimal Rank",
    "metric_value": minimal_rank,
    "instances_tested": 1,
    "conjecture_holds": None,
    "counterexample": ""
}

if __name__ == "__main__":
    if len(sys.argv) > 1:
        seeds = [int(s) for s in sys.argv[1:]]
    else:
        # Default list of 30 primes
        primes = [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97]
        seeds = random.sample(primes, min(30, len(primes)))

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    if all(r["conjecture_holds"] is None for r in results):
        RESULT = "INCONCLUSIVE mapping_undefined"
    else:
        supported_count = sum(1 for r in results if r["conjecture_holds"])
        support_fraction = supported_count / len(results)

        if support_fraction >= 0.7 or any(r["minimal_rank"] <= 5 * r["depth"] for r in results):
            RESULT = f"SUPPORTED mean={sum(r['metric_value'] for r in results) / len(results)} std={sum(r['metric_value']**2 for r in results) / len(results)}"
        else:
            first_failing_seed = next((r['seed'] for r in results if not r['conjecture_holds']), None)

```

```

RESULT = f"FALSE counterexample=\"mapping_undefined\" first_failing_seed={first_fa

print(RESULT)

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means that the pre-registered support conditions could not be evaluated. | next: Run the test again with increased time limits to ensure it completes and produces results.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12948
2	propose	ollama_remote	glm4:latest	0	0	11381
3	preregistration	ollama_remote	glm4:latest	0	0	6268
4	novelty	ollama_remote	glm4:latest	0	0	4667
5	novelty	ollama_remote	glm4:latest	0	0	6846
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16217
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7012
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8011
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9918
10	judge	ollama_remote	glm4:latest	0	0	16058

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 99325 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/04eb145028b1.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice

3. The full LLM transcripts are in `research/audit/04eb145028b1.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/04eb145028b1.tar.gz` (if generated)